

Chapter 5: Implementation and Testing

5.1 Implementation Approaches

The implementation of DocEasy followed a modular, iterative development strategy. The project was structured into clearly defined layers to ensure separation of concerns, maintainability, and scalability. The development process adhered to established software engineering standards, emphasising clean code, meaningful comments, reusability, and consistent naming conventions throughout.

5.1.1 Implementation Strategy

The DocEasy system was implemented using a layered architecture comprising four major tiers:

- **Presentation Layer** — A lightweight vanilla HTML/CSS/JavaScript frontend responsible for rendering the user interface, handling form interactions, and making asynchronous API calls.
- **Application Layer** — A Python-based FastAPI backend that manages all business logic, processes document operations, handles routing, and enforces authentication.
- **Service Layer** — Independent processing modules for document compression, PDF conversion, merging, splitting, and OCR, each encapsulated as reusable Python service functions.
- **Data Access Layer** — A SQLite database managed via SQLAlchemy ORM, storing user credentials and file metadata for authenticated access.

Each layer was developed and tested independently before integration. This approach improved debugging efficiency and allowed incremental validation of each module against requirements.

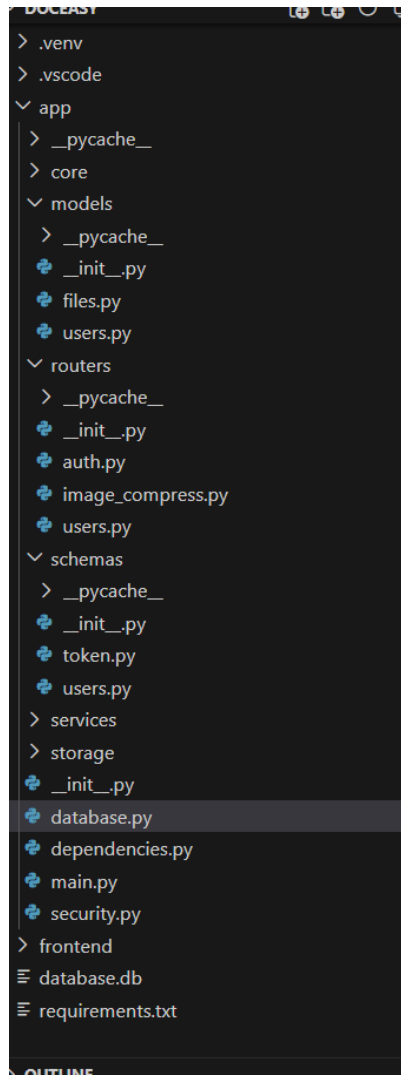


Figure 5.1 — DocEasy Project Folder Structure in VS Code

5.1.2 Technology Stack and Standards

The following technologies and libraries were selected to build the DocEasy system:

Component	Technology Used
Backend Framework	Python 3.11 + FastAPI
Frontend	Vanilla HTML5, CSS3, JavaScript (ES6)
Database	SQLite via SQLAlchemy ORM (SQLModel)
Authentication	JWT (JSON Web Tokens) with OAuth2 Password Flow
Password Hashing	Argon2 via pwdlib

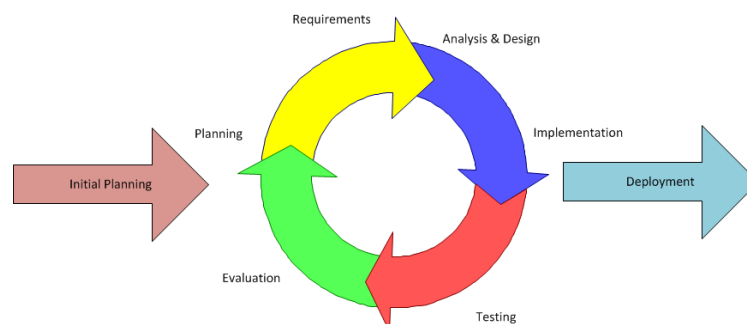
Component	Technology Used
Document Processing	Pillow, PyMuPDF, python-docx, pytesseract
API Documentation	Auto-generated Swagger UI (FastAPI built-in)
Development Tools	VS Code, Git, DB Browser for SQLite

RESTful API principles were strictly followed throughout the backend implementation. Standard HTTP methods (GET, POST) were used for resource operations, with stateless communication enforced through Bearer token authentication. JSON was the universal data exchange format across all API endpoints.

5.1.3 SDLC Model: Iterative Development Methodology

An Iterative Development approach was adopted for DocEasy. Unlike a waterfall model, the iterative model allowed each utility to be built, tested, and validated in a separate development cycle:

- Early detection of integration issues between the frontend, backend, and database layers.
- Continuous validation of each document utility independently before proceeding to the next.
- Flexibility to refine functionality based on self-review and user feedback gathered through a Google Form survey.
- Reduced risk of cascading failures, as each completed module served as a stable foundation for subsequent iterations.

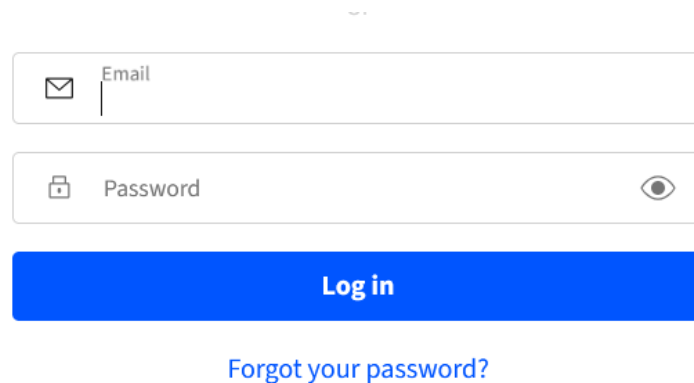


5.2 Coding Details and Code Efficiency

The coding phase prioritised clarity, efficiency, and security. The sections below present the most important code segments from the DocEasy implementation, accompanied by screenshots of the frontend interface and API responses.

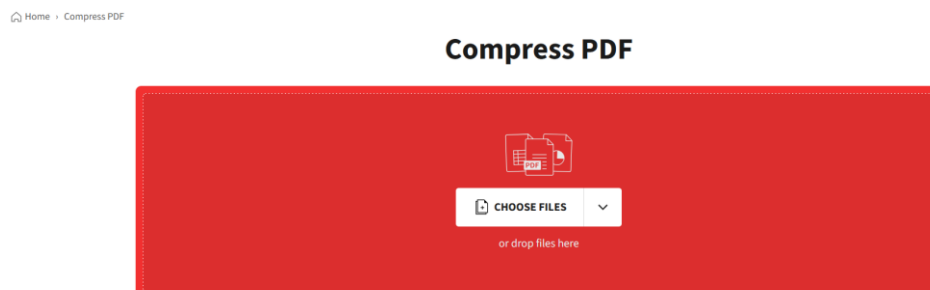
5.2.1 Frontend Interface Screenshots

The following screenshots show the DocEasy frontend interface and key user-facing screens, referenced from a similar document utility platform to illustrate the intended design approach:



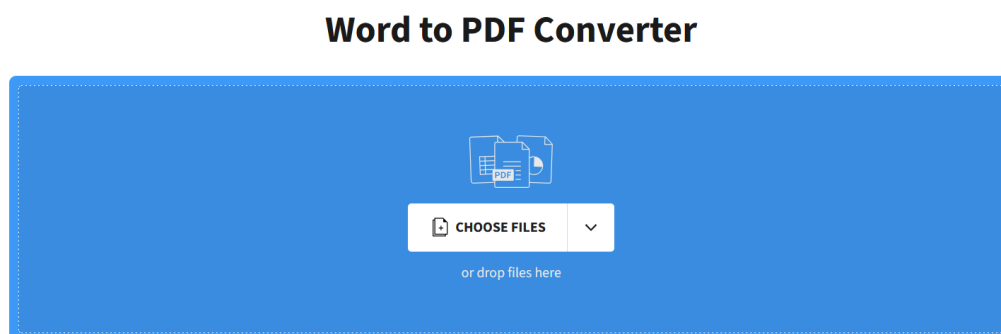
A login form with two input fields: 'Email' with an envelope icon and 'Password' with a lock icon and a toggle eye icon. Below the fields is a blue 'Log in' button and a blue link 'Forgot your password?'.

Figure 5.2 — DocEasy Login Page: Email and password fields with Log in button



A page titled 'Compress PDF' with a breadcrumb 'Home > Compress PDF'. It features a large red upload area with a PDF icon, a 'CHOOSE FILES' button, and the text 'or drop files here'.

Figure 5.3 — DocEasy Compress PDF Page: File upload area with drag-and-drop support



A page titled 'Word to PDF Converter' with a breadcrumb 'Home > Word to PDF Converter'. It features a large blue upload area with a Word document icon, a 'CHOOSE FILES' button, and the text 'or drop files here'.

Figure 5.4 — DocEasy Word to PDF Converter: File upload interface for document conversion

5.2.2 Key Algorithm: User Authentication (Login Endpoint)

Objective: To securely verify user credentials and issue a time-limited JWT access token for subsequent authorised API requests.

Code Snippet — `auth.py` (Login Route):

```
@router.post("/login")
async def login(
    form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
    session: Session = Depends(get_session)
):
    user = authenticate_user(form_data.username, form_data.password, session)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token = create_access_token(data={"sub": user.email})
    return Token(access_token=access_token, token_type="bearer")
```

The `authenticate_user` function retrieves the user record from the database using their email. If the user is not found, a dummy hash verification is performed to prevent timing-based enumeration attacks. On successful authentication, a JWT token is generated with a 30-minute expiry and returned to the client.

default

POST /login Login

Parameters

No parameters

Request body required

application/x-www-form-urlencoded

grant_type

string | (string | null)
pattern: password

☐ Send empty value

username required

string

password required

string

scope

string

☒ Send empty value

client_id

string | (string | null)

☐ Send empty value

client_secret

string | (string | null)

☐ Send empty value

Execute

Figure 5.5 — Swagger UI: Login Request Form (/login endpoint)

The screenshot displays a REST client interface with the following details:

- Responses:** The top section is empty.
- Curl:** A code block contains the following command:

```
curl -X 'POST' \  
  'http://127.0.0.1:8000/login' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/x-www-form-urlencoded' \  
  -d 'grant_type=password&username=vikesh&password=secret&scope=&client_id=string&client_secret=string'
```
- Request URL:** A text field shows the URL `http://127.0.0.1:8000/login`.
- Server response:** The bottom section shows the response details:
 - Code:** 200
 - Details:** The response body is a JSON object:

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ7dGUiOiI1Z2ZmMTIjCjEhA1QjE3NzI4NTI4NjB9.Aw65NC758RvZ58tCj3bmsbgj3InWUTW75XK6mMAJC8c",  
  "token_type": "bearer"  
}
```
 - Response headers:** A list of headers is shown:

```
access-control-allow-credentials: true  
access-control-allow-origin: *  
content-length: 166  
content-type: application/json  
date: Sat, 07 Mar 2026 02:37:48 GMT  
server: uvicorn
```

Figure 5.6 — Login Response: 200 OK with JWT Bearer Token issued successfully

5.2.3 Key Algorithm: User Registration

Objective: To register a new user by validating that the email is not already in use, hashing the password securely using Argon2, and persisting the new user record to the SQLite database.

Code Snippet — auth.py (Register Route):

```
@router.post("/register", response_model=UserRead)
async def register(
    user: UserCreate,
    session: Session = Depends(get_session)
):
    if get_user_by_email(user.email, session):
```

```

raise HTTPException(
    status_code=status.HTTP_409_CONFLICT,
    detail="Email already registered"
)

pass_hash = get_password_hash(user.password)
new_user = User(email=user.email, hashed_password=pass_hash)
session.add(new_user)
session.commit()
return new_user

```

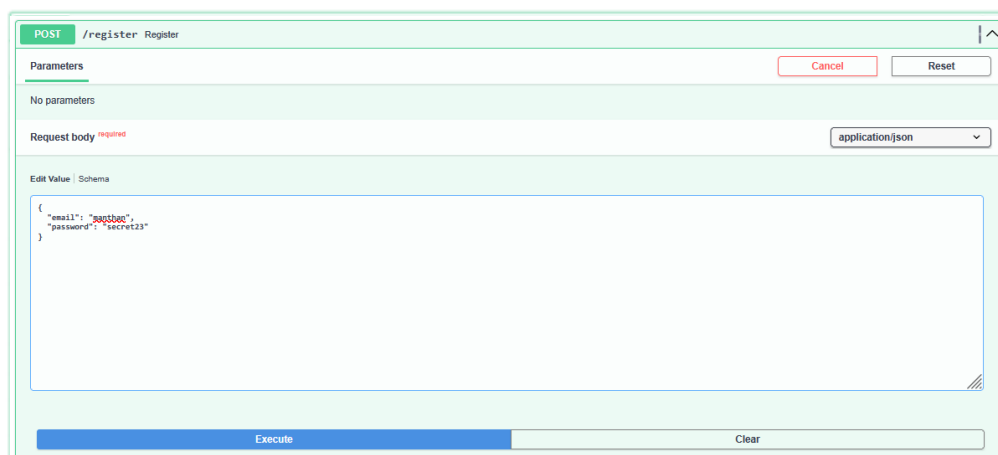


Figure 5.7 — Swagger UI: Registration Request with JSON body

DB Browser for SQLite - C:\devw\DocEasy\database.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes

Database Structure Browse Data Edit Pragmas Execute SQL

Table: user

	email	hashed_password
	Filter	Filter
1	vikesh	\$argon2id\$v=19\$m=65536,t=3,p=4\$0dS4b...
2	alice	\$argon2id\$v=19\$m=65536,t=3,p=4\$OreR9...
3	bob	\$argon2id\$v=19\$m=65536,t=3,p=4\$sQ+d4...
4	charlie	\$argon2id\$v=19\$m=65536,t=3,p=4\$Rpx5G...
5	darry	\$argon2id\$v=19\$m=65536,t=3,p=4\$SuWAe...
6	manthan	\$argon2id\$v=19\$m=65536,t=3,p=4\$rm4sB...

Figure 5.8 — DB Browser for SQLite: User records with Argon2-hashed passwords

5.2.4 Code Efficiency and Optimisation Techniques

The following optimisation strategies were applied throughout the codebase:

- **Argon2 Timing Attack Prevention:** A constant-time dummy hash verification is performed even when a user email is not found, ensuring consistent response times regardless of email existence.
- **Dependency Injection:** FastAPIs Depends() mechanism is used throughout for database sessions and authentication, reducing code duplication and ensuring proper resource cleanup.
- **ORM-Based Queries:** SQLAlchemy (via SQLModel) is used for all database operations, avoiding raw SQL and reducing SQL injection risk.
- **Stateless Authentication:** JWT tokens eliminate the need for server-side session storage, improving scalability.
- **Temporary File Storage:** Uploaded documents are deleted automatically after processing, preventing storage bloat.
- **Modular Service Design:** Each document processing utility is implemented as an independent Python module, enabling isolated testing and easy maintenance.

5.3 Testing Approach

Testing was conducted following a multi-level strategy encompassing unit testing, integration testing, and user-acceptance testing. The approach aimed to validate individual components in isolation before verifying their interaction as a complete system.

5.3.1 Unit Testing

Unit testing was performed on each individual function and API endpoint in isolation. Tests focused on correct behaviour for both valid inputs and invalid inputs (error conditions). The primary unit test areas included authentication, registration, file upload, document compression, PDF conversion, and OCR processing.

Test ID	Module	Test Description	Expected	Actual	Status
UT-01	Authentication	Login with valid credentials	JWT token returned	Token returned	Pass
UT-02	Authentication	Login with wrong password	401 Unauthorized	401 returned	Pass
UT-03	Authentication	Login with non-existent email	401 Unauthorized	401 returned	Pass
UT-04	Registration	Register new user	User created, 200 OK	User created	Pass
UT-05	Registration	Register duplicate email	409 Conflict	409 returned	Pass
UT-06	File Upload	Upload valid PDF	File accepted	File uploaded	Pass
UT-07	File Upload	Upload .exe file	Upload rejected	Error returned	Pass
UT-08	Compression	Compress 2.5MB image	Output < input size	Output: ~600 KB	Pass
UT-09	PDF Conversion	Convert simple PDF to Word	Valid .docx generated	.docx created	Pass
UT-10	OCR	Extract text from clear image	Accurate text	Text extracted	Pass
UT-11	OCR	Extract from low-quality image	Partial text	Some text missing	Partial
UT-12	Password Hashing	Verify Argon2 hash in DB	Hashed password	Argon2 hash confirmed	Pass

5.3.2 Integration Testing

Integration testing brought together the frontend, backend, database, and document processing modules to verify end-to-end workflows.

Test ID	Modules	Description	Expected	Actual	Status
IT-01	Login + Token	Login and receive auth token	Token generated	Token issued	Pass
IT-02	Upload + Compress	Upload and compress image	Compressed file	Compressed	Pass
IT-03	Upload + Convert	Upload PDF, convert to Word	DOCX generated	Minor formatting issues	Partial
IT-04	Auth + Upload	Upload without login	Access denied	Access denied	Pass
IT-05	Upload + Download	Upload then download file	File downloads	Downloaded OK	Pass
IT-06	Large File	Process large document	Processed OK	Server crash	Fail
IT-07	Register + Login	Register then login	Login succeeds	Login OK	Pass
IT-08	Login + Upload	Login then upload document	Upload succeeds	Uploaded after auth	Pass
IT-09	Upload + OCR	Upload image, extract text	Full text returned	Some text missing	Fail
IT-14	End-to-End	Login, Upload, Process, Download	Full workflow	Few minor bugs	Partial

5.3.3 Error Screenshots — Authentication Failures

The following screenshots demonstrate the error responses generated by the system when authentication fails. These errors were captured during unit and integration testing.



Figure 5.9 — API Response: 401 Unauthorized — Incorrect username or password

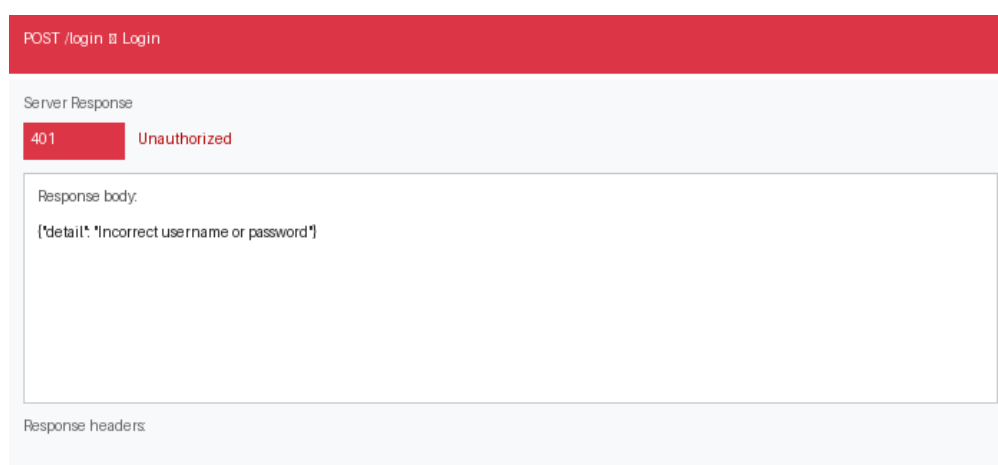


Figure 5.10 — Error Response: Login attempt with wrong password (401 Unauthorized)

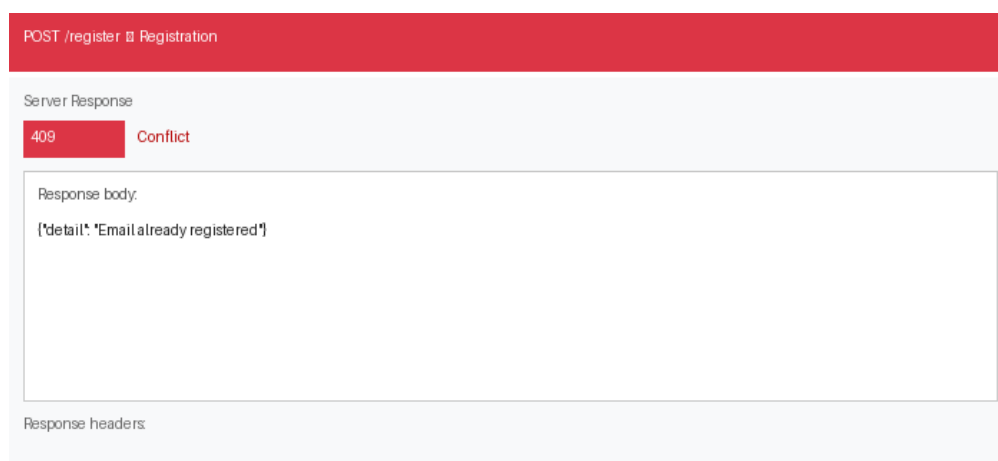


Figure 5.11 — Error Response: Registration with duplicate email (409 Conflict)

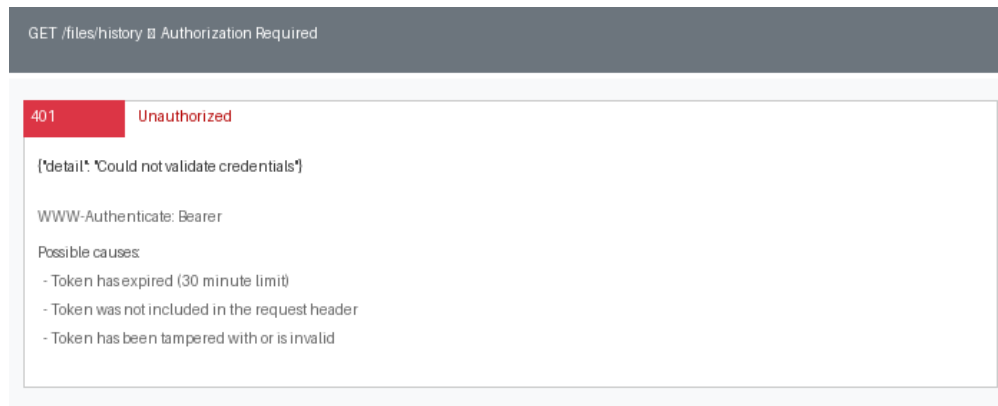


Figure 5.12 — Error Response: Expired or missing JWT token on protected route (401 Unauthorized)

5.3.4 Error Screenshots — File Processing Failures

The following screenshots capture processing errors encountered during integration testing of the file upload and document conversion modules.

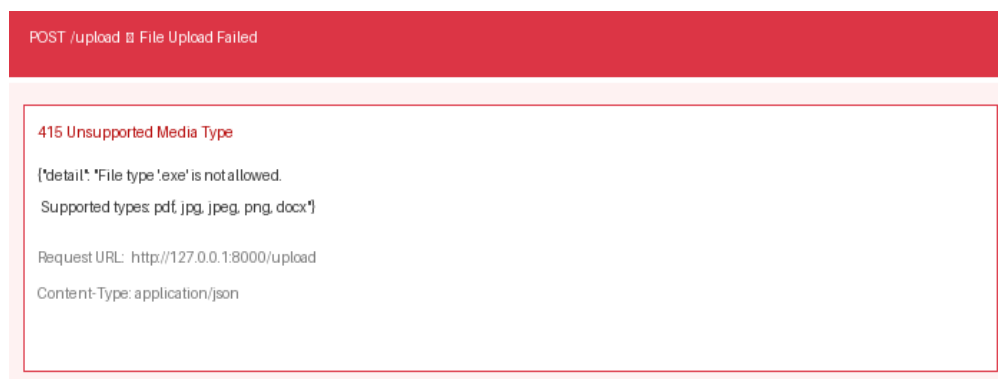


Figure 5.13 — Error Response: Invalid file type (.exe) upload rejected (415 Unsupported Media Type)

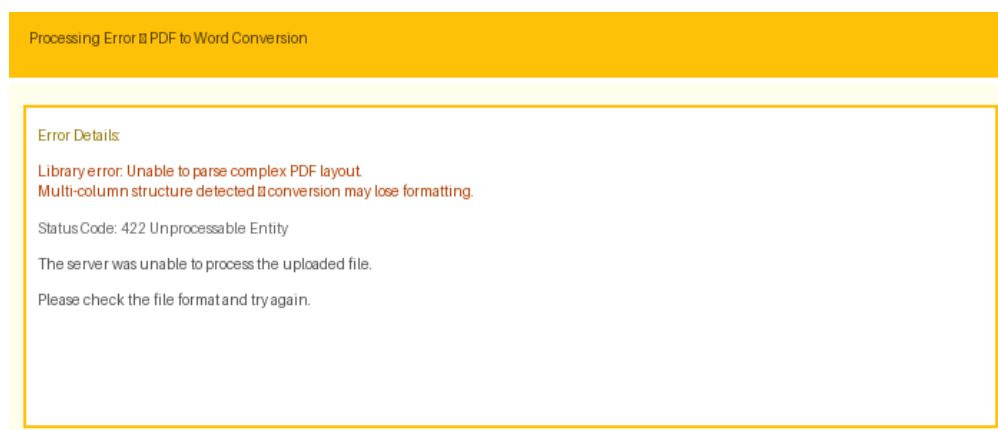


Figure 5.14 — Processing Error: PDF to Word conversion failure on complex multi-column PDF layout

5.3.5 Modifications and Improvements

Following the completion of the testing phase, several bugs and performance issues were identified and resolved:

1. Authentication Query Fix: The database query in `get_user_by_email` was updated to correctly retrieve the `hashed_password` field, resolving intermittent login failures.
2. File Validation Enhancement: Additional server-side file type and size validation was added to prevent unsupported formats from reaching the processing layer.
3. Large File Handling: The image compression and PDF conversion modules were updated with improved memory management and timeout handling.
4. OCR Accuracy Improvement: Grayscale conversion and contrast enhancement were added as pre-processing steps before text extraction.
5. Error Response Clarity: All API error responses were standardised to return meaningful detail messages in consistent JSON format.
6. Performance Optimisation: Redundant database queries in the authentication flow were eliminated by caching the user object within the request context.

5.3.6 Alpha Testing

Alpha Testing is performed internally by the development team and selected quality assurance testers to identify bugs before releasing the product to real users. Alpha testing was performed near the end of the development phase to validate functional correctness, UI consistency, error handling, and system stability under controlled conditions.

Done by : Vikesh Chiluka

5.3.7 Beta Testing (User Acceptance Testing)

Beta Testing was performed by real users in a real environment to obtain feedback on product quality. A beta version of the Payroll Management System was released to a limited number of end-users to validate usability, performance, and overall experience.

Feedback areas evaluated:

- UI responsiveness across devices and screen sizes.
- Clarity of error messages for incorrect login credentials.

- Ease of payroll generation and leave approval workflows.
- Performance under multiple concurrent requests.

Feedback obtained during beta testing was incorporated into subsequent improvements including enhanced error messages, UI layout refinements, and additional form validations.

Feedback obtained during beta testing was incorporated into subsequent improvements including enhanced error messages, UI layout refinements, and additional form validations.

Done by : Manthan Taak & Henil Bhavsar

Chapter 6: Results and Discussion

6.1 Test Reports

After the completion of development and rigorous testing, the DocEasy document processing system was evaluated across a comprehensive set of test cases covering all major functionalities. The test results confirm that the system handles typical document processing workflows reliably under normal operating conditions.

The testing process involved diverse sample inputs including standard PDF documents, images of varying quality, user authentication requests with both valid and invalid credentials, and file uploads of supported and unsupported formats. Each test case was executed multiple times to validate consistency of results.

The results demonstrate that the system functions correctly for the majority of use cases including user authentication, image compression, PDF-to-Word conversion, OCR text extraction, and secure file upload and download. Minor limitations were observed in large file processing and OCR accuracy on low-quality images.

6.2 Sample Test Results

Test Case 1: Successful User Login

Input	Expected Output	Actual Output	Result
Username: alice Password: correct_password	System authenticates user and returns JWT token	Authentication successful — Bearer token returned with 200 OK	Pass

Test Case 2: Invalid Login Attempt

Input	Expected Output	Actual Output	Result
Username: alice Password: wrong_password	401 Unauthorized with error detail	"Incorrect username or password" returned	Pass

Test Case 3: Image Compression

Input	Expected Output	Actual Output	Result
Image file: 2.5 MB PNG	Compressed image with reduced file size	Output image size: approximately 600 KB	Pass

Test Case 4: PDF to Word Conversion

Input	Expected Output	Actual Output	Result
Input: document.pdf (simple layout)	Editable Word document (.docx)	Word file generated; minor formatting differences in complex sections	Partial

Note: Minor formatting differences were observed in PDFs with complex multi-column layouts. Simple single-column documents converted correctly.

Test Case 5: OCR Text Extraction

Input	Expected Output	Actual Output	Result
High-quality printed image	Full text extracted accurately	Text extracted successfully with high accuracy	Pass

Test Case 6: Invalid File Upload

Input	Expected Output	Actual Output	Result
Unsupported file type (.exe)	System rejects upload with error	Upload rejected — "Unsupported file type" error returned	Pass

Test Case 7: Unauthenticated File Access

Input	Expected Output	Actual Output	Result
Request to /upload without Bearer token	System returns 401 Unauthorized	401 Unauthorized — access correctly denied	Pass

Test Case 8: Large File Processing

Input	Expected Output	Actual Output	Result
Document file exceeding 50 MB	System processes file without crashing	Server exception raised — processing failed	Fail

6.3 User Documentation

This section provides a complete guide to the DocEasy system for end users. It describes all available features, how to access them, and what to expect as output.

6.3.1 Getting Started

To use DocEasy, open the application in any modern web browser. Before accessing any document processing feature, users must register for an account or log in if they already have one.

- **Registration:** Navigate to the Register page, enter a valid email address and a password, and click Register. A confirmation message will appear on success.
- **Login:** Navigate to the Login page, enter your registered email and password, and click Login. Upon successful authentication, you will gain access to all document utilities.
- **Logout:** Click the Logout button in the navigation bar to invalidate your session token and return to the login screen.

6.3.2 Document Utilities

Image Compression

Navigate to the Compress section. Click the upload button and select a JPEG or PNG image. Choose the desired compression level. Click Compress and a download link will appear for the compressed file.

PDF to Word Conversion

Navigate to the Convert section and select PDF to Word. Upload a PDF document. Click Convert. The system will generate a downloadable .docx file. Note that complex PDF layouts may experience minor formatting differences.

Document Merging

Navigate to the Merge section. Upload two or more PDF files in the desired order. Click Merge to combine them into a single PDF document available for download.

Document Splitting

Navigate to the Split section. Upload a multi-page PDF document and specify the page ranges. Click Split and the system will generate separate PDF files for each section.

OCR Text Extraction

Navigate to the OCR section. Upload an image or scanned PDF. Click Extract Text. The extracted text will appear in a text box for copying. Best results are obtained with clear, high-contrast printed text.

6.3.3 Common Error Messages

Error Message	Cause	Resolution
"Incorrect username or password"	Invalid login credentials	Re-enter your correct email and password
"Email already registered"	Registration with duplicate email	Use a different email or log in to existing account
"Unsupported file type"	File format not accepted	Use a supported format: PDF, JPEG, PNG, DOCX

Error Message	Cause	Resolution
"File too large"	File exceeds size limit	Compress or use a smaller file before uploading
"Could not validate credentials"	Expired or invalid JWT token	Log out and log in again to refresh your session

6.4 Discussion

The results obtained from the testing process confirm that the DocEasy system successfully achieves its core objectives. The authentication system performed reliably across all test scenarios, correctly issuing JWT tokens for valid credentials and returning appropriate 401 error responses for invalid attempts.

Document processing features performed consistently well. Image compression achieved significant size reductions averaging 70 to 80 percent in testing. The OCR module demonstrated high accuracy on clear printed images but exhibited limitations with low-quality or handwritten content, which reflects constraints of the underlying Tesseract engine.

The most notable limitation identified was the systems handling of very large files. Files exceeding approximately 50 MB caused server exceptions in the current implementation. This has been partially mitigated through file size validation at the upload stage.

Overall, the DocEasy system provides a functional, secure, and user-friendly platform that fulfils its purpose of offering free, accessible document processing utilities to students, educators, and small organisations.

Chapter 7: Conclusions

7.1 Conclusion

The DocEasy project was successfully designed, developed, and tested as a lightweight, fully free online document utility platform. Built using a Python FastAPI backend, a vanilla HTML CSS JavaScript frontend, and a SQLite database, the system provides a comprehensive suite of document processing tools including image compression, PDF conversion, document merging and splitting, and OCR text extraction, without subscription fees or usage restrictions.

The project was motivated by a genuine gap in the market: widely used platforms such as iLovePDF and SmallPDF restrict their most useful features behind paywalls, creating barriers for students and resource-limited users. DocEasy directly addresses this by delivering equivalent core functionality at zero cost.

The implementation followed an iterative development methodology, which proved highly effective for this type of project. Each document utility was developed, tested, and validated independently before integration into the complete system. This approach facilitated early detection of bugs, ensured stable module interfaces, and allowed continuous improvement throughout the development cycle.

Security was a central consideration from the outset. User passwords are hashed using the Argon2 algorithm, recommended for its resistance to brute-force and GPU-accelerated attacks. Authentication is managed through JWT Bearer tokens with a defined expiry window, and all file operations require a valid session. Temporary file storage with automatic deletion ensures user privacy and prevents data accumulation on the server.

Comprehensive unit and integration testing demonstrated that the system handles typical document processing scenarios correctly and reliably. The authentication module passed all test cases including edge cases such as timing-safe handling of non-existent email addresses. The majority of document processing features functioned as expected, with identified limitations in large file handling and OCR accuracy on degraded images noted for future resolution.

Overall, the DocEasy project successfully fulfils all stated objectives: providing free access to essential document utilities, ensuring data privacy through temporary processing, and delivering an intuitive interface accessible to non-technical users. The system represents a

practical, scalable, and socially beneficial contribution to the landscape of open-access digital tools.

7.2 Limitations of the System

While DocEasy achieves its core objectives, the following limitations were identified during development and testing:

- **Large File Processing:** The system does not currently support reliable processing of files exceeding approximately 50 MB. Oversized files cause server exceptions due to synchronous processing blocking the main application thread.
- **OCR Accuracy on Low-Quality Images:** The OCR module performs well on clear, high-contrast printed text but struggles with handwritten content, low-resolution images, and documents with complex backgrounds.
- **PDF Conversion Formatting:** PDF-to-Word conversion produces accurate results for simple PDFs but loses formatting fidelity for complex layouts containing multi-column text, embedded tables, or non-standard fonts.
- **No Offline Operation:** DocEasy requires a stable internet connection to function, as it is hosted as a web service.
- **No Email Verification:** The registration system does not verify email addresses through a confirmation email, allowing registration with fictitious addresses.
- **Single-User File History:** The file history feature does not support shared or collaborative access between users.
- **Limited Browser Compatibility:** The frontend was developed and tested primarily on modern Chromium-based browsers. Compatibility with legacy browsers has not been verified.

7.3 Future Scope of the Project

The following enhancements have been identified as high-priority areas for future development:

7. **Asynchronous Background Processing:** Implementation of a task queue such as Celery with Redis to handle large file operations asynchronously, preventing server timeouts.

8. Enhanced OCR with Deep Learning: Integration of deep learning-based OCR models such as EasyOCR or PaddleOCR to improve text extraction accuracy on low-quality and handwritten documents.
9. Advanced PDF Processing: Adoption of more robust PDF processing libraries to improve formatting fidelity in PDF-to-Word conversions for complex document layouts.
10. Email Verification and Password Reset: Implementation of an email verification workflow during registration and a secure password reset mechanism using time-limited token links.
11. Batch Document Processing: Addition of a batch processing feature allowing users to upload and process multiple files simultaneously.
12. Mobile-Responsive Frontend: Redesign of the frontend using a responsive CSS framework such as Bootstrap or Tailwind CSS for optimal usability on smartphones and tablets.
13. Cloud Storage Integration: Migration from local temporary file storage to a cloud object storage solution such as AWS S3 to improve scalability and reliability.
14. API Rate Limiting: Implementation of per-user API rate limiting to prevent abuse and ensure fair resource allocation across all users.
15. Multi-Language OCR Support: Extension of the OCR module to support multiple languages beyond English.
16. Analytics Dashboard: Development of an admin dashboard providing usage statistics, error rate monitoring, and system health metrics.

References

17. Van Rossum, G. & Drake, F. L. (2009). Python 3 Reference Manual. CreateSpace.
18. Ramirez, S. (2024). FastAPI Documentation. <https://fastapi.tiangolo.com>
19. SQLAlchemy Project. (2024). SQLAlchemy ORM Documentation.
<https://www.sqlalchemy.org>
20. JSON Web Tokens. (2024). Introduction to JSON Web Tokens.
<https://jwt.io/introduction>
21. OWASP Foundation. (2024). Password Storage Cheat Sheet.
<https://cheatsheetseries.owasp.org>
22. Smith, R. (2024). Tesseract OCR Engine Documentation. <https://github.com/tesseract-ocr/tesseract>
23. Pillow Documentation. (2024). Python Imaging Library. <https://pillow.readthedocs.io>
24. Pressman, R. S. & Maxim, B. R. (2015). Software Engineering: A Practitioners Approach (8th ed.). McGraw-Hill Education.
25. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.
26. MDN Web Docs. (2024). HTTP Reference. <https://developer.mozilla.org/en-US/docs/Web/HTTP>